

UFC

UNIVERSIDADE FEDERAL DO CEARÁ – UFC
CENTRO DE TECNOLOGIA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

Gateway DCIM: monitoramento de data center

Sistemas distribuídos com multicast, UDP, gRPC e TCP, todas as mensagens em Protocol Buffers

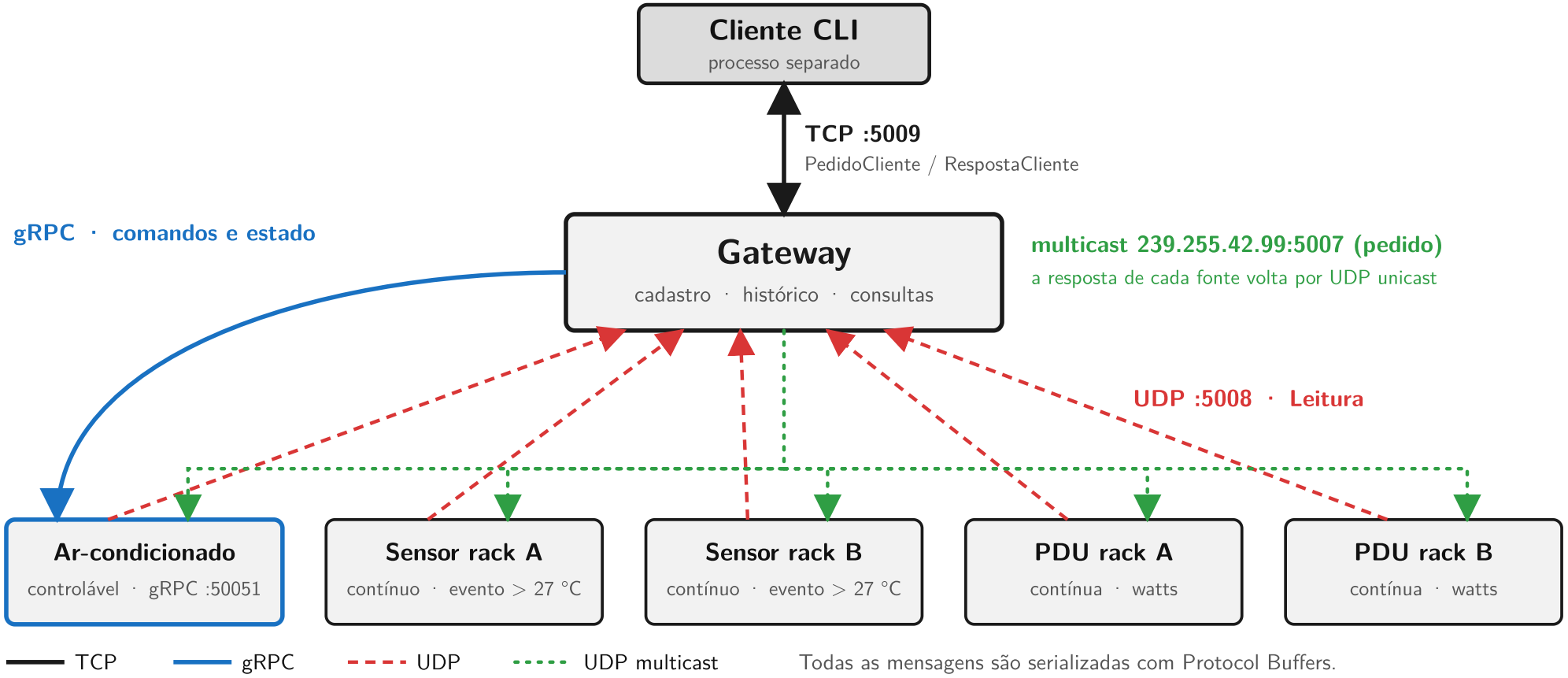
Antonio Marcos Justino Carneiro · matrícula **511911**
Augusto

TI0151 – Sistemas Distribuídos (2026.2) · Professor: Paulo A. L. Rego

FORTALEZA, CE · 2026

Domínio e arquitetura

Domínio: **data center (DCIM)**. O gateway reúne as fontes; o operador consulta e comanda pelo cliente.



Limiar de alerta de **27 °C** (topo da faixa recomendada, ASHRAE TC 9.9) · unidades °C e W · 5 fontes na demonstração: 1 ar-condicionado, 2 sensores, 2 PDUs.

Linguagem, frameworks e bibliotecas

Uma única linguagem, sem frameworks de aplicação: sockets da biblioteca padrão, Protobuf e gRPC.

Camada	Tecnologia
Linguagem	Java 21
Build	Maven 3.8 · `protobuf-maven-plugin` gera o código Protobuf e gRPC a cada `mvn compile`
Serialização	Protocol Buffers (proto3), `protobuf-java` 3.25.5 · um único `datacenter.proto` para todos os processos
Invocação remota	gRPC Java 1.68.0 (`grpc-netty-shaded` , `grpc-protobuf` , `grpc-stub`)
Sockets e threads	`java.net` : `MulticastSocket` , `DatagramSocket` , `ServerSocket` , `Socket` · threads da biblioteca padrão

Processos: `Gateway` , `ArCondicionado` , `SensorTemperatura` , `Pdu` , `Cliente` · código: github.com/JustinoCarneiro/gateway-dcim-sd

Descoberta

UDP multicast

O gateway não conhece os IPs das fontes: pergunta ao grupo `239.255.42.99:5007`; cada fonte responde por unicast.

```
// gateway → grupo multicast
message DescobertaSolicitacao {
  string gateway_ip         = 1;
  uint32 gateway_porta_udp  = 2;
  uint32 nonce              = 3;
}

// fonte → gateway (unicast)
message DescobertaResposta {
  string fonte_id    = 1;
  TipoFonte tipo     = 2;
  string ip          = 3;
  uint32 porta       = 4;  // gRPC (0 = sem)
  bool controlavel   = 5;
  string rack        = 6;
  uint32 intervalo_ms = 7;
  optional EstadoAr estado_inicial = 8;
  uint32 nonce       = 9;
}
```

- Ao iniciar, o gateway envia o pedido e o **repete a cada 30 s**, para achar fontes que subiram depois ou perderam o pedido (UDP).
- O pedido leva o **IP e a porta UDP do gateway**: a fonte aprende para onde enviar as leituras, sem configuração.
- A resposta vai por unicast ao remetente; o **nonce** descarta respostas de rodadas anteriores.

Leituras

UDP

Um datagrama = uma mensagem `Leitura`, enviada à porta 5008 do gateway.

```
message Leitura {
  string fonte_id      = 1;
  TipoFonte tipo       = 2;
  string rack          = 3;
  Grandeza grandeza    = 4;  // °C ou W
  double valor         = 5;
  int64 timestamp_ms   = 6;  // hora da medição
  bool evento          = 7;  // envio imediato
  uint32 intervalo_ms  = 8;  // período de envio
  uint64 seq           = 9;
  optional EstadoAr estado_ar = 10;
}
```

- **Periódico** a cada 2 s e **evento imediato** ao cruzar 27 °C, subindo ou descendo (`evento = true`).
- UDP não garante entrega: o gateway confere o `seq` e registra **perda, duplicação e reordenação**.
- O ar-condicionado anexa `estado_ar` e, desligado, continua enviando: sem leituras = falha.
- O gateway usa a **hora de chegada**, não o `timestamp_ms` da fonte (relógios diferentes).

Serviço gRPC do ar-condicionado

gRPC

A fonte controlável é o servidor; o gateway é o cliente. Definição em `datacenter.proto`.

```
service ArCondicionado {
  rpc Configurar(ConfigurarRequest) returns (EstadoAr);
  rpc ObterEstado(Vazio)            returns (EstadoAr);
}

message ConfigurarRequest {
  optional bool   ligado         = 1;
  optional double setpoint_c     = 2;  // 10..35
  optional uint32 ventilador_pct = 3;  // 0..100
  optional uint32 intervalo_ms   = 4;  // 500..60000
}

message EstadoAr {
  bool ligado = 1;  double setpoint_c = 2;
  uint32 ventilador_pct = 3;  uint32 intervalo_ms = 4;
}
```

- **Atualização parcial:** com `optional`, campo ausente = não alterar.
- **Idempotente:** o comando carrega valores absolutos; repetir após um timeout é seguro.
- **Validação na fonte:** valor fora da faixa devolve `INVALID_ARGUMENT`, repassado ao cliente.
- **Prazo de 3 s** por chamada; fonte fora do ar devolve `UNAVAILABLE`.

Cliente ↔ gateway e consultas

TCP

TCP é um fluxo de bytes: cada mensagem Protobuf vai com 4 bytes de tamanho na frente.




```
message PedidoCliente {
  oneof pedido {
    ListarFontesRequest listar   = 1;
    ComandoRequest      comando  = 2;  // fonte_id + config
    ConsultaRequest     consulta = 3;  // tipo + janela_s
    HistoricoRequest    historico = 4; // fonte_id + limite
  }
}
message RespostaCliente {
  bool   ok   = 1;
  string erro = 2;
  oneof resposta {
    ListarFontesResponse listar   = 3;
    ComandoResponse      comando  = 4;
    ConsultaResponse     consulta = 5;
    HistoricoResponse    historico = 6;
  }
}
```

- **Conteúdo inválido:** `ok = false` e a conexão continua. **Tamanho acima de 1 MiB:** conexão fechada.
- Uma thread por cliente conectado.

3 consultas (janela padrão 300 s)

- `media` : média das temperaturas dos sensores
- `rack` : rack com a maior média de temperatura
- `pico` : maior potência das PDUs e qual PDU

Histórico em CSV

- `historico [id] [n]` : últimas leituras do CSV, mesmo após reiniciar

Falhas, concorrência e limitações

Situação	Comportamento
Fonte sem leituras por mais de **3 intervalos**	`INATIVA` ; volta a `ATIVA` na próxima leitura
Comando a fonte que caiu	gRPC `UNAVAILABLE` ; o gateway segue
Comando fora da faixa	`INVALID_ARGUMENT` da fonte, repassado ao cliente
UDP perdido, duplicado ou fora de ordem	Detectado pelo `seq` e registrado no log
TCP inválido ou com 2 GiB	`ok = false` / conexão fechada; o gateway segue

Concorrência

- Gateway: thread UDP, thread TCP (uma por cliente) e agenda de descoberta.
- Estado compartilhado em `Cadastro` (`synchronized`).

Limitações

- Gateway é ponto único de falha; sem TLS nem autenticação.
- O detector por tempo não distingue fonte caída de rede caída.